

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Case Study

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO3 – Precision	Assessment:	Assessment Tool 2 – Case Study
Weightage:	30% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	Harish M	Reg. No.:	192424014
Section / Batch:	CSE-AI / 3 rd year/2024	Date:	27/08/2026

Code of Conduct

I Harish M / 192424014 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

Instructions

- Read all three case studies carefully before attempting the questions.
- Provide appropriate justifications and interpretations for every computed result.
- Use NLP concepts, algorithms, and terminology wherever applicable.
- Diagrams, flowcharts, tables, or mathematical formulations may be used to support your answers.
- Duration: 30 minutes.

Section / Topic	Focus	Case Study
N-Gram Language Models and Smoothing Techniques	Bigram and Trigram probability computation, Maximum Likelihood Estimation (MLE), Backoff models, Deleted Interpolation, Entropy calculation, uncertainty analysis, real-time next-word prediction	Case Study 1 – Smart Mobile Keyboard Prediction System
Part-of-Speech (POS)	Penn Treebank tagset, contextual ambiguity resolution, rule-based tagging, stochastic tagging	Case Study 2 – AI-Powered Customer

Tagging and Word Classes	(HMM), emission and transition probabilities, word class identification, chatbot language understanding	Support Chatbot
Transformation-Based Tagging (Brill Tagger)	POS tag correction, transformation rules, contextual tagging, linguistic validation, tag sequence refinement, ambiguity handling	Case Study 3 – News Analytics and POS Tag Correction System
Corpus Statistics and Probabilistic Analysis	Word frequency counting, corpus-based probability estimation, entropy-based uncertainty measurement, tagging confidence evaluation	Case Study 3 – News Analytics and POS Tag Correction System
Integrated Syntactic Analysis and NLP Applications	Application of language models, POS tagging, probabilistic methods, corpus analysis, ambiguity resolution, and decision-making in real-world NLP systems	Case Studies 1, 2, and 3

Annexure A – Case Study

CASE STUDY 1: E-Commerce Smart Search and Product Prediction System

An e-commerce company is developing an AI-powered search and autocomplete system that predicts the next word when customers enter product-related queries. The language model is trained on the corpus:

"machine learning improves business, machine learning enables automation, machine learning drives innovation."

The development team uses **Bigram and Trigram Language Models, Backoff Models, Deleted Interpolation, and Entropy Analysis** to improve search prediction and handle unseen word sequences.

The following statistics are obtained from the corpus:

- $C(\text{machine}) = 3$
- $C(\text{machine learning}) = 3$
- $C(\text{learning improves}) = 1$
- $C(\text{learning enables}) = 1$
- $C(\text{learning drives}) = 1$
- $C(\text{learning improves business}) = 1$

The interpolation weights are:

- $\lambda_1 = 0.5$ (Trigram)
- $\lambda_2 = 0.3$ (Bigram)

- $\lambda_3 = 0.2$ (Unigram)

The prediction probabilities after the word "**learning**" are:

- $P(\text{improves}) = 0.33$
- $P(\text{enables}) = 0.33$
- $P(\text{drives}) = 0.33$

Questions

1. Compute the Maximum Likelihood Estimate (MLE) of the Bigram Probability **$P(\text{learning} \mid \text{machine})$** . Interpret how this probability can influence the next-word prediction of an e-commerce search system.
2. A customer enters the unseen sequence "**machine learning transforms**". Apply the Backoff Model and estimate the probability of the word "**transforms**" using lower-order N-grams. Explain why backoff is useful when customers enter previously unseen search queries.
3. Using Deleted Interpolation, calculate the probability of the sequence "**machine learning improves**" using the given interpolation weights. Explain how combining trigram, bigram, and unigram probabilities improves robustness when training data is sparse.
4. Calculate the Entropy of the prediction distribution **$P(\text{improves}) = 0.33$, $P(\text{enables}) = 0.33$, $P(\text{drives}) = 0.33$** . Interpret the entropy value and explain what it indicates about uncertainty in the search prediction system.
5. Develop a Python program using **NLTK** for N-gram generation, **NumPy** for probability computations, and the **Math** library for entropy calculation. The program should construct Unigram, Bigram, and Trigram models from the given corpus, compute the MLE Bigram probability, estimate an unseen sequence using Backoff, calculate the Deleted Interpolation probability using the given weights, compute entropy of the prediction distribution, and display all intermediate calculations and the final next-word prediction in a structured format.

CASE STUDY 2: AI-Powered Hospital Appointment Chatbot

A healthcare organization develops an AI-powered chatbot that assists patients in booking appointments. The chatbot performs **Part-of-Speech tagging** before identifying user intentions and generating responses.

Consider the following user inputs:

1. "**Book an appointment with the doctor.**"
2. "**The book contains medical information.**"

The chatbot uses the **Penn Treebank POS Tagset** and an **HMM-based probabilistic POS tagger** to determine the grammatical role of ambiguous words.

For the word "**book**", the system is given:

- $P(\text{book} \mid \text{VB}) = 0.7$
- $P(\text{book} \mid \text{NN}) = 0.3$
- $P(\text{Start} \rightarrow \text{VB}) = 0.6$

- $P(\text{Start} \rightarrow \text{NN}) = 0.4$

The system must determine whether **"book"** functions as a Verb (VB) or Noun (NN) depending on its context.

Questions

1. Assign appropriate **Penn Treebank POS tags** to every word in both sentences. Justify the POS tag assigned to **"book"** using the grammatical context of each sentence.
2. Using the given HMM probabilities, compute the likelihood of **"book"** being tagged as a Verb (VB) and as a Noun (NN). Determine the most probable tag and explain why the probabilistic model favors that tag at the beginning of a sentence.
3. Compare **Rule-Based POS Tagging** and **Stochastic HMM Tagging** for resolving the ambiguity of the word **"book"**. Discuss the advantages and limitations of both approaches in a healthcare chatbot.
4. Evaluate how standardized POS tagsets such as the **Penn Treebank Tagset** can support downstream NLP tasks such as **intent detection, entity identification, and response generation** in healthcare chatbots.
5. Develop a Python program using **NLTK** to tokenize the given sentences and assign Penn Treebank POS tags. Implement a simple **HMM-based POS tagging calculation** using the given emission and transition probabilities. The program should calculate the probability of **"book"** being tagged as VB and NN, determine the most likely tag, display the tokenized sentences, POS-tagged output, intermediate probability calculations, and final tagging results in a structured format.

CASE STUDY 3: Financial News POS Tag Correction and Uncertainty Analysis

A financial news analytics company processes thousands of financial news articles every day. The system initially assigns POS tags using a statistical POS tagger and subsequently applies a **Transformation-Based Tagger (Brill Tagger)** to correct common tagging errors.

The sentence processed by the system is:

"Market growth drives investment."

The initial POS tags produced by the system are:

- market/NN
- growth/NN
- drives/NNS
- investment/NN

A learned transformation rule states:

Change NNS to VBZ if the preceding word is tagged as NN.

The system also performs corpus-based frequency analysis and entropy-based uncertainty evaluation.

The corpus contains the following word frequencies:

1. market – 500 occurrences
2. growth – 350 occurrences
3. drives – 180 occurrences
4. investment – 420 occurrences

Questions

1. Apply the given transformation rule to the initial POS-tag sequence. Display the corrected POS-tag sequence and explain why the transformation is linguistically appropriate.
2. Analyze the initial tagging error for "**drives**". Explain why the word should receive the **VBZ** tag in this sentence and discuss how the surrounding grammatical context helps identify the correct tag.
3. Using the given corpus statistics, calculate the **relative frequency/probability** of each word. Explain how corpus frequency information can support probabilistic POS tagging and language processing.
4. Calculate the **entropy of the word-frequency distribution** before and after applying the transformation rule. Explain what entropy indicates about uncertainty in the tagging or prediction process.
5. Develop a Python program using **NLTK** for initial POS tagging and implement a simple **Transformation-Based Tagger** that applies the rule "**Change NNS to VBZ if the preceding word is tagged as NN.**" The program should tokenize the sentence, generate initial POS tags, apply the transformation rule, calculate word-frequency probabilities from the given corpus statistics, calculate entropy using the Python **math** library, and display the initial tags, corrected tags, frequency distribution, entropy values, and final POS-tagging results in a structured format.

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with	Good analysis with logical	Basic analysis with limited	Weak or no analysis; lacks

		strong reasoning and insights	reasoning	depth	reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Q. No. / Criteria Level	Understanding of Case	Application of Concepts	Analysis	Solution Design	Clarity	Sub. Total	Total %
1	4	3	4	4	3		92
2	4	4	4	4	3		
3	4	3	3	4	4		

Faculty In-charge	Course Coordinator	Program Director
Dr. T Kumaragurubaran		
Signature: T. Kumaragurubaran	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Introduction

This report presents the solutions for the three case studies in Assessment Tool 2. The cases cover N-gram language models and smoothing, POS tagging with contextual ambiguity and HMM probabilities, and transformation-based POS correction with corpus statistics and entropy.

The uploaded assessment requires appropriate justifications and interpretations for computed results and permits mathematical formulations, tables and diagrams to support the answers.

Case Study Overview

Case Study	Application	Major NLP Concepts
1	E-Commerce Smart Search and Product Prediction	Bigram MLE, Backoff, Deleted Interpolation, Entropy
2	AI-Powered Hospital Appointment Chatbot	Penn Treebank POS, HMM, emission/transition probabilities
3	Financial News POS Tag Correction	Transformation-Based Tagging, corpus frequency, entropy

Case Study 1 – E-Commerce Smart Search and Product Prediction System

The system predicts the next word in product-related search queries. The supplied corpus is: 'machine learning improves business, machine learning enables automation, machine learning drives innovation.'

The case provides $C(\text{machine})=3$, $C(\text{machine learning})=3$, $C(\text{learning improves})=1$, $C(\text{learning enables})=1$, $C(\text{learning drives})=1$ and $C(\text{learning improves business})=1$. The interpolation weights are $\lambda_1=0.5$ for trigram, $\lambda_2=0.3$ for bigram and $\lambda_3=0.2$ for unigram.

Question 1 – Bigram MLE

The Maximum Likelihood Estimate is:

$$P(\text{learning} \mid \text{machine}) = C(\text{machine learning}) / C(\text{machine})$$

$$= 3 / 3$$

$$= 1.00$$

Therefore, the MLE probability is 1.00 or 100%. In the supplied training corpus, every occurrence of 'machine' is followed by 'learning'. A high conditional probability makes 'learning' a very strong next-word candidate after 'machine', so an autocomplete system would rank it highly.

Question 2 – Backoff for 'machine learning transforms'

The sequence is unseen. A backoff system first checks the trigram, then the relevant bigram, and finally the unigram probability.

Trigram: $P(\text{transforms} \mid \text{machine, learning}) = 0$ (unseen)

Bigram: $P(\text{transforms} \mid \text{learning}) = 0$ (unseen)

Unigram: $P(\text{transforms}) = 0$ (word absent from corpus)

Thus, with the exact corpus supplied in the case study, the estimated probability of 'transforms' is 0 because the word itself does not occur in the training vocabulary. Backoff is still useful in general because many unseen higher-order sequences can obtain a non-zero lower-order probability. For an entirely unseen word, an additional unknown-word or vocabulary strategy would be required.

Question 3 – Deleted Interpolation

For 'machine learning improves', the supplied probabilities are approximately 0.33 for the trigram and 0.33 for the bigram. The corpus contains 15 word tokens, so $P(\text{improves}) = 1/15 = 0.0667$.

$P(\text{improves} \mid \text{machine, learning}) = 1/3 \approx 0.33$

$P(\text{improves} \mid \text{learning}) = 1/3 \approx 0.33$

$P(\text{improves}) = 1/15 \approx 0.0667$

$P = \lambda_1 P_3 + \lambda_2 P_2 + \lambda_3 P_1$

$= 0.5(0.33) + 0.3(0.33) + 0.2(0.0667)$

≈ 0.2533

Therefore, the interpolated probability is approximately 0.2533 using the supplied rounded N-gram probabilities. Interpolation is more robust for sparse data because it combines evidence from different orders rather than relying only on a single N-gram.

Question 4 – Entropy

The prediction distribution is $P(\text{improves})=0.33$, $P(\text{enables})=0.33$ and $P(\text{drives})=0.33$. Using the values exactly as supplied:

$$H = -\sum p(x) \log_2 p(x)$$

$$= -[0.33 \log_2(0.33) + 0.33 \log_2(0.33) + 0.33 \log_2(0.33)]$$

$$\approx 1.5848 \text{ bits}$$

The value is close to the maximum entropy of a three-way uniform distribution, $\log_2(3) \approx 1.585$ bits. This indicates high uncertainty: the system has almost equal confidence in the three possible next words.

Question 5 – Python Program

```
import nltk
import numpy as np
import math
from collections import Counter

nltk.download("punkt", quiet=True)

corpus = """
machine learning improves business.
machine learning enables automation.
machine learning drives innovation.
"""

# Tokenization
sentences = [s.strip().lower().split(".") for s in corpus.split(".") if s.strip()]
tokens = []
for s in sentences:
    tokens.extend(s[0].split())

# N-gram counts
unigrams = Counter(tokens)
bigrams = Counter(zip(tokens[:-1], tokens[1:]))
trigrams = Counter(
    zip(tokens[:-2], tokens[1:-1], tokens[2:])
)

print("UNIGRAM COUNTS")
print(unigrams)

print("\nBIGRAM COUNTS")
print(bigrams)

print("\nTRIGRAM COUNTS")
print(trigrams)

# MLE Bigram probability
p_learning_machine = (
    bigrams[("machine", "learning")]
    / unigrams["machine"]
)

print("nP(learning | machine)=", p_learning_machine)

# Backoff for transforms
trigram_count = trigrams[("machine", "learning", "transforms")]
bigram_count = bigrams[("learning", "transforms")]
transform_count = unigrams["transforms"]
```

```

print("\nBACKOFF CALCULATION")
print("Trigram count =", trigram_count)
print("Bigram count =", bigram_count)
print("Unigram count =", transform_count)

if trigram_count > 0:
    p_backoff = trigram_count / bigrams[("machine", "learning")]
elif bigram_count > 0:
    p_backoff = bigram_count / unigrams["learning"]
elif transform_count > 0:
    p_backoff = transform_count / len(tokens)
else:
    p_backoff = 0

print("P(transforms) by backoff =", p_backoff)

# Deleted Interpolation
lambda1 = 0.5 # trigram
lambda2 = 0.3 # bigram
lambda3 = 0.2 # unigram

p3 = trigrams[("machine", "learning", "improves")] / bigrams[("machine", "learning")]
p2 = bigrams[("learning", "improves")] / unigrams["learning"]
p1 = unigrams["improves"] / len(tokens)

p_interp = (
    lambda1 * p3 +
    lambda2 * p2 +
    lambda3 * p1
)

print("\nDELETED INTERPOLATION")
print("Trigram =", p3)
print("Bigram =", p2)
print("Unigram =", p1)
print("Final probability =", p_interp)

# Entropy
prediction_probs = np.array([0.33, 0.33, 0.33])
entropy = -sum(
    p * math.log2(p)
    for p in prediction_probs
    if p > 0
)

print("\nENTROPY =", entropy, "bits")

# Final prediction
next_words = {
    "improves": 0.33,
    "enables": 0.33,
    "drives": 0.33
}

print("\nNEXT-WORD PREDICTION")
for word, probability in next_words.items():
    print(word, "->", probability)

```

```

UNIGRAM COUNTS
Counter({'machine': 3, 'learning': 3, 'improves': 1, 'business': 1, 'enables': 1, 'automation': 1, 'drives': 1, 'innovation': 1})

BIGRAM COUNTS
Counter({'machine', 'learning': 3, ('learning', 'improves'): 1, ('improves', 'business'): 1, ('business', 'machine'): 1, ('learning', 'enables'): 1, ('enables', 'automation'): 1, ('automation', 'drives'): 1, ('drives', 'innovation'): 1})

TRIGRAM COUNTS
Counter({'machine', 'learning', 'improves': 1, ('learning', 'improves', 'business'): 1, ('improves', 'business', 'machine'): 1, ('business', 'machine', 'learning'): 1, ('machine', 'learning', 'enables'): 1, ('learning', 'enables', 'automation'): 1, ('enables', 'automation', 'drives'): 1, ('automation', 'drives', 'innovation'): 1})

P(learning | machine) = 1.0

BACKOFF CALCULATION
Trigram count = 0
Bigram count = 0
Unigram count = 0
P(transforms) by backoff = 0

DELETED INTERPOLATION
Trigram = 0.3333333333333333
Bigram = 0.3333333333333333
Unigram = 0.08333333333333333
Final probability = 0.2833333333333333

ENTROPY = 1.5834674497121084 bits

NEXT-WORD PREDICTION
improves -> 0.33
enables -> 0.33
drives -> 0.33

```

Case Study 2 – AI-Powered Hospital Appointment Chatbot

The chatbot performs POS tagging before identifying patient intent and generating responses. The case uses the Penn Treebank tagset and an HMM-based probabilistic POS tagger to resolve the ambiguity of the word 'book'.

Question 1 – Penn Treebank POS Tags

Sentence 1: 'Book an appointment with the doctor.'

Word	Penn Treebank Tag	Reason
Book	VB	Verb: action of scheduling/reserving an appointment
an	DT	Determiner
appointment	NN	Singular common noun
with	IN	Preposition
the	DT	Determiner
doctor	NN	Singular common noun
.	.	Punctuation

Sentence 2: 'The book contains medical information.'

Word	Penn Treebank Tag	Reason
The	DT	Determiner

book	NN	Noun: an object containing information
contains	VBZ	Third-person singular present verb
medical	JJ	Adjective
information	NN	Singular common noun
.	.	Punctuation

The word 'book' changes from VB in the first sentence to NN in the second because its grammatical role is determined by context. In the first sentence it denotes an action; in the second it denotes an object.

Question 2 – HMM Probability

$$P(\text{book} \mid \text{VB}) \times P(\text{Start} \rightarrow \text{VB}) = 0.7 \times 0.6 = 0.42$$

$$P(\text{book} \mid \text{NN}) \times P(\text{Start} \rightarrow \text{NN}) = 0.3 \times 0.4 = 0.12$$

The initial HMM calculation therefore favors VB because $0.42 > 0.12$. The relative score of VB is 3.5 times the NN score. However, this is an initial-state calculation using only the probabilities provided. A complete sentence-level HMM would also use later transition/emission evidence, so the second sentence can be correctly interpreted as NN when its surrounding context is included.

Question 3 – Rule-Based vs Stochastic HMM

Aspect	Rule-Based	Stochastic HMM
Basis	Hand-crafted linguistic rules	Probabilities learned from tagged data
Strength	Transparent and easy to explain	Can model contextual likelihood
Ambiguity	Depends on manually designed rules	Uses emission and transition probabilities
Data Need	Low training-data requirement	Requires a tagged corpus
Healthcare Use	Predictable for controlled language	Better for varied patient language
Limitation	Rules may become complex	Needs reliable training data and probability estimates

For a healthcare chatbot, a hybrid approach can be useful: deterministic rules can handle known clinical or appointment phrases, while a stochastic model can resolve ambiguous words using context.

Question 4 – Importance of Penn Treebank Tags

1. Intent detection can use grammatical roles to distinguish actions from objects.
2. Entity identification benefits from separating nouns, adjectives and other word classes.
3. Response generation can use syntactic information when constructing or selecting responses.
4. A standardized tagset provides consistent labels across NLP components.
5. Contextual ambiguity can be represented more systematically than with raw tokens alone.

Question 5 – Python Program

```
import nltk
from nltk import word_tokenize

nltk.download("punkt", quiet=True)

sentences = [
    "Book an appointment with the doctor.",
    "The book contains medical information."
]

print("TOKENIZATION AND POS TAGGING")
print("=" * 60)

for sentence in sentences:
    tokens = word_tokenize(sentence)
    tags = nltk.pos_tag(tokens)

    print("\nSentence:", sentence)
    print("Tokens:", tokens)
    print("POS tags:")
    for word, tag in tags:
        print(f'{word:15s} -> {tag}')

# Given HMM probabilities
emission_VB = 0.7
emission_NN = 0.3

transition_VB = 0.6
transition_NN = 0.4

score_VB = emission_VB * transition_VB
score_NN = emission_NN * transition_NN

print("\nHMM CALCULATION")
print("P(book | VB) =", emission_VB)
print("P(Start -> VB) =", transition_VB)
print("VB score =", score_VB)

print("\nP(book | NN) =", emission_NN)
print("P(Start -> NN) =", transition_NN)
print("NN score =", score_NN)

if score_VB > score_NN:
    print("\nMost probable initial tag: VB")
else:
    print("\nMost probable initial tag: NN")
```

```
print("\nCONTEXTUAL INTERPRETATION")
print("Sentence 1: Book -> VB")
print("Sentence 2: book -> NN")
```

```
TOKENIZATION AND POS TAGGING
=====

Sentence: Book an appointment with the doctor.
Tokens: ['Book', 'an', 'appointment', 'with', 'the', 'doctor', '.']
POS tags:
Book      -> NNP
an        -> DT
appointment -> NN
with      -> IN
the       -> DT
doctor    -> NN
.         -> .

Sentence: The book contains medical information.
Tokens: ['The', 'book', 'contains', 'medical', 'information', '.']
POS tags:
The       -> DT
book      -> NN
contains  -> VBZ
medical   -> JJ
information -> NN
.         -> .

HMM CALCULATION
=====
P(book | VB) = 0.7
P(Start -> VB) = 0.6
VB score = 0.42

P(book | NN) = 0.3
P(Start -> NN) = 0.4
NN score = 0.12

Most probable initial tag: VB

CONTEXTUAL INTERPRETATION
=====
Sentence 1: Book -> VB
Sentence 2: book -> NN
```

Case Study 3 – Financial News POS Tag Correction and Uncertainty Analysis

The financial news system initially tags 'Market growth drives investment.' as market/NN, growth/NN, drives/NNS, investment/NN. A learned transformation rule states: 'Change NNS to VBZ if the preceding word is tagged as NN.'

Question 1 – Apply Transformation Rule

Initial: market/NN growth/NN drives/NNS investment/NN

Rule: Change NNS to VBZ if preceding word is NN

Corrected: market/NN growth/NN drives/VBZ investment/NN

The rule applies because 'drives' is tagged NNS and its preceding word 'growth' is tagged NN. The corrected VBZ tag represents a third-person singular present verb, which is grammatically appropriate in the sentence: 'Market growth drives investment.'

Question 2 – Why 'drives' is VBZ

The sentence has the structure subject + verb + object. 'Market growth' is the noun phrase acting as the subject, 'drives' expresses the action, and 'investment' is the object. Since the subject is singular, the present-tense verb takes the third-person singular form 'drives', represented by VBZ.

Question 3 – Corpus Frequency Probabilities

Total frequency:

$$500 + 350 + 180 + 420 = 1450$$

Word	Frequency	Relative Probability
market	500	$500/1450 = 0.3448$
growth	350	$350/1450 = 0.2414$
drives	180	$180/1450 = 0.1241$
investment	420	$420/1450 = 0.2897$

Corpus frequency provides empirical evidence about how common words are in the available data. A probabilistic NLP system can use these frequencies as part of word likelihood estimation, although frequency alone cannot determine POS because the same word can have different grammatical roles.

Question 4 – Entropy Before and After Transformation

The supplied frequency distribution is a distribution over words. The transformation changes the POS tag of 'drives' but does not change the four word frequencies. Therefore, the word-frequency entropy is unchanged before and after the transformation.

$$H = -\sum p(w) \log_2 p(w)$$

$$p(\text{market}) = 500/1450 = 0.3448$$

$$p(\text{growth}) = 350/1450 = 0.2414$$

$$p(\text{drives}) = 180/1450 = 0.1241$$

$$p(\text{investment}) = 420/1450 = 0.2897$$

$$H_{\text{before}} \approx 1.9161 \text{ bits}$$

$$H_{\text{after}} \approx 1.9161 \text{ bits}$$

Thus, the transformation improves the POS-tag sequence without changing the word-frequency distribution. The entropy of the supplied word distribution remains 1.9161 bits. If entropy were instead defined over POS-tag probabilities or tag-sequence probabilities, a different distribution would be needed; the case study does not provide those additional statistics.

Question 5 – Python Program

```
import nltk
import math
from collections import Counter

nltk.download("punkt", quiet=True)

sentence = "Market growth drives investment."

# Initial tags supplied by the case
initial_tags = [
    ("market", "NN"),
    ("growth", "NN"),
    ("drives", "NNS"),
    ("investment", "NN")
]

print("INITIAL POS TAGS")
for word, tag in initial_tags:
    print(f"{word:15s} -> {tag}")

# Transformation-Based Rule
corrected_tags = initial_tags.copy()

for i in range(1, len(corrected_tags)):
    current_word, current_tag = corrected_tags[i]
    previous_word, previous_tag = corrected_tags[i - 1]

    if current_tag == "NNS" and previous_tag == "NN":
        corrected_tags[i] = (current_word, "VBZ")

print("\nCORRECTED POS TAGS")
for word, tag in corrected_tags:
    print(f"{word:15s} -> {tag}")

# Frequency data
frequency = {
    "market": 500,
    "growth": 350,
    "drives": 180,
    "investment": 420
}

total = sum(frequency.values())

probabilities = {
    word: count / total
    for word, count in frequency.items()
}

print("\nWORD FREQUENCY DISTRIBUTION")
```



```

for word, count in frequency.items():
    print(
        f"{word:15s} "
        f"count={count:4d} "
        f"probability={probabilities[word]:.4f}"
    )

# Entropy
entropy_before = -sum(
    p * math.log2(p)
    for p in probabilities.values()
    if p > 0
)

# Word frequencies are unchanged after POS transformation
entropy_after = entropy_before

print("\nENTROPY ANALYSIS")
print("Entropy before transformation =", entropy_before, "bits")
print("Entropy after transformation =", entropy_after, "bits")

print("\nFINAL RESULT")
print("The transformation changes drives/NNS to drives/VBZ.")
print("The word-frequency entropy remains unchanged.")

```

```

INITIAL POS TAGS
market      -> NN
growth      -> NN
drives      -> NNS
investment   -> NN

CORRECTED POS TAGS
market      -> NN
growth      -> NN
drives      -> VBZ
investment   -> NN

WORD FREQUENCY DISTRIBUTION
market      count= 500 probability=0.3448
growth      count= 350 probability=0.2414
drives      count= 180 probability=0.1241
investment   count= 420 probability=0.2897

ENTROPY ANALYSIS
Entropy before transformation = 1.916090490949558 bits
Entropy after transformation  = 1.916090490949558 bits

FINAL RESULT
The transformation changes drives/NNS to drives/VBZ.
The word-frequency entropy remains unchanged.

```

Conclusion

The three case studies demonstrate practical applications of N-gram models, smoothing and interpolation, POS tagging, HMM probability estimation, transformation-based tagging, and entropy.

The calculations show how language-model probabilities support next-word prediction, how contextual probabilities can help resolve POS ambiguity, and how transformation rules can correct systematic tagging errors.

The results also show the importance of interpreting probabilities in their application context. Backoff is useful when a higher-order N-gram is missing, interpolation improves robustness by combining multiple probability estimates, HMM tagging uses emission and transition evidence, and transformation rules refine an initially generated tag sequence.